Repository

Persist users behind collection-like API

Season 4 · Enterprise · Intermediate

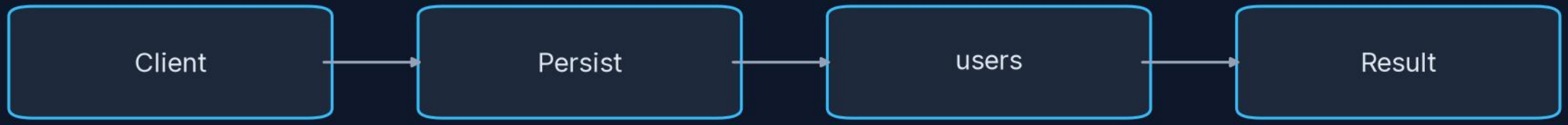
Today's Hook

- Problem: Persist users behind collection-like API
- Naive pain cue: Optional.empty
- We will refactor to Repository with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: Persist users behind collection-like API
- Constraints: solve «Persist users behind collection-like API» with Repository — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Persist users behind collection-like API
- Success looks like: null

Structure



Repository: Persist users behind collection-like API

Real-World Scenario

- Running example: Persist users behind collection-like API
- Key types: User, UserRepository, InMemoryUserRepository
- Happy-path stdout cue: null
- Failure-path stdout cue: Optional.empty

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.repository.problem;

import java.util.*;

public final class NaiveUsers {
  private final Map<String,String> db=new HashMap<>();
  public void insert(String id,String email){ db.put(id,email); }
  public String get(String id){ return db.get(id); } // null
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Persist users behind collection-like API
- Without Repository, call sites absorb the wrong details.
- Failure mode to show on camera: Optional.empty
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.repository.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record User(String id, String email){}
interface UserRepository {
  void save(User u); Optional<User> findById(String id); List<User> findAll();
}
final class InMemoryUserRepository implements UserRepository {
  private final Map<String,User> db=new LinkedHashMap<>();
  public void save(User u){ db.put(u.id(), u); }
  public Optional<User> findById(String id){ return Optional.ofNullable(db.get(id)); }
  public List<User> findAll(){ return List.copyOf(db.values()); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.repository.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Repository: users collection API ===");
    System.out.println("--- naive ---");
var n=new NaiveUsers(); n.insert("1","a@b.c"); System.out.println(n.get("missing"));
System.out.println("--- repository ---");
UserRepository repo=new InMemoryUserRepository(); repo.save(new User("1","a@b.c"));
System.out.println(repo.findById("1")); System.out.println(repo.findById("x"));
  }
}
```

Walkthrough Output

```
=== Repository: users collection API ===
--- naive ---
null
--- repository ---
Optional[User[id=1, email=a@b.c]]
Optional.empty
```

Step-by-Step Trace

- Match stdout to stages — look for: null
- Then the injected failure: Optional.empty
- Re-run: java -cp ... 26-repository ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.enterprise.repository.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record User(String id, String email){}
interface UserRepository {
  void save(User u); Optional<User> findById(String id); List<User> findAll();
}
final class InMemoryUserRepository implements UserRepository {
  private final Map<String,User> db=new LinkedHashMap<>();
  public void save(User u){ db.put(u.id(), u); }
  public Optional<User> findById(String id){ return Optional.ofNullable(db.get(id)); }
  public List<User> findAll(){ return List.copyOf(db.values()); }
}
```

Trade-offs

- Use Repository when «Persist users behind collection-like API» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Repository in product language.

Related Patterns

- Name the pattern usually confused with Repository and separate the forces.
- Sketch the roles for «Persist users behind collection-like API» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Persist users behind collection-like API
- Naive pain → Repository boundary → Demo stdout.
- Remember the failure cue: Optional.empty
- Break a ProblemTest assertion, watch it fail, restore it.